

.NET Concepts for Backend Developer

Real Observability Dashboard inside our own app.

Building here is a production-grade Observability Platform — exactly what strong backend & system engineers do.

Feature	Benefit
Admin observability	Real production-level monitoring
Role protection	Security compliance
Trace visualization	Debug microservices fast
Business insight	Know which APIs are slow

💬 Interview Killer Line

“We implemented an internal observability dashboard where only admins can view logs, metrics, and distributed traces. This helped our team debug performance issues, detect failures early, and analyze request flow across microservices.”

📦 FINAL SYSTEM ARCHITECTURE:

Angular 20 Admin Dashboard

|
| JWT (Admin)
v

.NET 8 Observability API

|
|-- OpenTelemetry (Tracing + Metrics)
|-- Serilog → SQL Logs
|
|--> Prometheus (Metrics)
|--> Jaeger (Traces)
|--> SQL Server (Logs)

Admin UI Sidebar:

Observability

├— Logs
├— Metrics
└— Tracing

.NET Concepts for Backend Developer

TECHNOLOGY STACK:

Layer	Tech
Frontend	Angular 20
Backend	.NET 8 Web API
Auth	JWT + Role = Admin
Logs	Serilog → SQL Server
Metrics	OpenTelemetry → Prometheus
Tracing	OpenTelemetry → Jaeger
Database	SQL Server

PHASED BUILD PLAN

Phase	What we build
Phase 1	.NET 8 API + JWT + Admin Role
Phase 2	OpenTelemetry Tracing
Phase 3	Metrics + Prometheus
Phase 4	Logging + SQL
Phase 5	Angular Admin UI
Phase 6	Connect UI to backend
Phase 7	Dashboard polishing

PHASE 1 — Backend Foundation (.NET 8 + JWT + Admin)

1 Create Solution

```
dotnet new webapi -n Observability.API
```

```
cd Observability.API
```

2 Add Required Packages

```
dotnet add package Microsoft.AspNetCore.Authentication.JwtBearer
```

```
dotnet add package Serilog.AspNetCore
```

```
dotnet add package Serilog.Sinks.MSSqlServer
```

.NET Concepts for Backend Developer

3 Create Admin User Model

Models/User.cs

```
public class User
{
    public int Id { get; set; }
    public string Username { get; set; } = "";
    public string Password { get; set; } = "";
    public string Role { get; set; } = "User";
}
```

4 JWT Configuration

Program.cs

```
using Microsoft.AspNetCore.Authentication.JwtBearer;
using Microsoft.IdentityModel.Tokens;
using System.Text;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(options =>
    {
        options.TokenValidationParameters = new TokenValidationParameters
        {
            ValidateIssuer = false,
            ValidateAudience = false,
            ValidateIssuerSigningKey = true,
            IssuerSigningKey =
                new SymmetricSecurityKey(Encoding.UTF8.GetBytes("SUPER_SECRET_KEY_123"))
        };
    });

builder.Services.AddAuthorization();

builder.Services.AddControllers();

var app = builder.Build();
```

.NET Concepts for Backend Developer

```
app.UseAuthentication();
```

```
app.UseAuthorization();
```

```
app.MapControllers();
```

```
app.Run();
```

5 Auth Controller (Admin Login)

Controllers/AuthController.cs

```
using Microsoft.AspNetCore.Mvc;
```

```
using Microsoft.IdentityModel.Tokens;
```

```
using System.IdentityModel.Tokens.Jwt;
```

```
using System.Security.Claims;
```

```
using System.Text;
```

```
[ApiController]
```

```
[Route("api/auth")]
```

```
public class AuthController : ControllerBase
```

```
{
```

```
    [HttpPost("login")]
```

```
    public IActionResult Login()
```

```
    {
```

```
        var claims = new[]
```

```
        {
```

```
            new Claim(ClaimTypes.Name, "admin"),
```

```
            new Claim(ClaimTypes.Role, "Admin")
```

```
        };
```

```
        var key = new SymmetricSecurityKey(Encoding.UTF8.GetBytes("SUPER_SECRET_KEY_123"));
```

```
        var token = new JwtSecurityToken(
```

```
            signingCredentials: new SigningCredentials(key, SecurityAlgorithms.HmacSha256),
```

.NET Concepts for Backend Developer

```
        claims: claims,
        expires: DateTime.UtcNow.AddHours(1)
    );

    return Ok(new
    {
        token = new JwtSecurityTokenHandler().WriteToken(token)
    });
}
```

Secure Admin API

Controllers/ObservabilityController.cs

```
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;

[Authorize(Roles = "Admin")]
[ApiController]
[Route("api/observability")]
public class ObservabilityController : ControllerBase
{
    [HttpGet("status")]
    public string GetStatus() => "Admin dashboard secured";
}
```

Test Phase 1

1. Call /api/auth/login → get token
2. Call /api/observability/status with token
3. Without token → **401 Unauthorized**

.NET Concepts for Backend Developer

PHASE 2 — OpenTelemetry Tracing + Jaeger

Goal:

Every request to your .NET API produces **trace data** viewable in a tracing UI (Jaeger).

Step 1 — Install Tracing Packages

Inside **Observability.API**:

```
dotnet add package OpenTelemetry
```

```
dotnet add package OpenTelemetry.Extensions.Hosting
```

```
dotnet add package OpenTelemetry.Instrumentation.AspNetCore
```

```
dotnet add package OpenTelemetry.Instrumentation.Http
```

```
dotnet add package OpenTelemetry.Exporter.Jaeger
```

Step 2 — Configure OpenTelemetry (Program.cs)

Open **Program.cs** and update:

```
using OpenTelemetry.Resources;
```

```
using OpenTelemetry.Trace;
```

```
builder.Services.AddOpenTelemetry()  
    .WithTracing(tracer =>  
    {  
        tracer  
            .SetResourceBuilder(  
                ResourceBuilder.CreateDefault()  
                    .AddService("Observability.API")  
                    .AddAspNetCoreInstrumentation()  
                    .AddHttpClientInstrumentation()  
                    .AddJaegerExporter();  
            });  
    });
```

Now **every HTTP request** is traced automatically.

Step 3 — Add Custom Business Trace

Create **Tracing/TracingService.cs**

.NET Concepts for Backend Developer

```
using System.Diagnostics;
```

```
public class TracingService
{
    private static readonly ActivitySource Activity =
        new("Observability.API");

    public void TrackOrder()
    {
        using var span = Activity.StartActivity("ProcessOrder");
        Thread.Sleep(200);
    }
}
```

Register it:

```
builder.Services.AddSingleton<TracingService>();
```

Use it inside controller:

```
[HttpGet("trace-test")]
public string TraceTest([FromServices] TracingService tracer)
{
    tracer.TrackOrder();
    return "Trace generated";
}
```



Step 4 — Run Jaeger (Tracing UI)

Create **docker-compose.yml**

```
version: '3'
```

```
services:
```

```
  jaeger:
```

```
    image: jaegertracing/all-in-one
```

```
    ports:
```

```
      - "16686:16686"
```

.NET Concepts for Backend Developer

- "6831:6831/udp"

Run:

docker compose up

Step 5 — Test Tracing

Run API:

dotnet run

Hit endpoints:

/api/observability/status

/api/observability/trace-test

Open browser:

http://localhost:16686

You will see:

Service: Observability.API

Trace timeline

Spans:

- HTTP GET

- ProcessOrder

This is **real distributed tracing**.

What You Now Have

Capability	Status
Automatic HTTP tracing	✓
Custom business spans	✓
Trace visualization	✓
Production tracing stack	✓

.NET Concepts for Backend Developer

PHASE 3 — Metrics + Prometheus

Goal:

Your .NET API will expose **live metrics** that Prometheus can scrape and store.

Step 1 — Install Metrics Packages

```
dotnet add package OpenTelemetry.Instrumentation.Runtime
```

```
dotnet add package OpenTelemetry.Exporter.Prometheus.AspNetCore
```

Step 2 — Configure Metrics in Program.cs

Update OpenTelemetry configuration:

```
builder.Services.AddOpenTelemetry()
    .WithTracing(tracer =>
    {
        tracer
            .SetResourceBuilder(ResourceBuilder.CreateDefault())
            .AddService("Observability.API")
            .AddAspNetCoreInstrumentation()
            .AddHttpClientInstrumentation()
            .AddJaegerExporter();
    })
    .WithMetrics(metrics =>
    {
        metrics
            .AddAspNetCoreInstrumentation()
            .AddRuntimeInstrumentation()
            .AddPrometheusExporter();
    });
```

Step 3 — Expose Metrics Endpoint

After building app:

```
app.UseOpenTelemetryPrometheusScrapingEndpoint();
```

.NET Concepts for Backend Developer

Now your API exposes:

`http://localhost:5000/metrics`

Step 4 — Run Prometheus

Update **docker-compose.yml**

version: '3'

services:

jaeger:

image: jaegertracing/all-in-one

ports:

- "16686:16686"

- "6831:6831/udp"

prometheus:

image: prom/prometheus

ports:

- "9090:9090"

volumes:

- ./prometheus.yml:/etc/prometheus/prometheus.yml

Create **prometheus.yml**

global:

scrape_interval: 5s

scrape_configs:

- job_name: "observability_api"

static_configs:

- targets: ["host.docker.internal:5000"]

Run:

`docker compose up`

Step 5 — Test Metrics

.NET Concepts for Backend Developer

Call some API endpoints.

Then open:

`http://localhost:9090`

Search metrics:

- `http_server_request_duration_seconds`
- `process_cpu_time_seconds`
- `dotnet_total_memory_bytes`

You now have **real-time system metrics**.

What You Now Have

Feature	Status
System metrics	✓
HTTP metrics	✓
Prometheus integration	✓
Enterprise monitoring	✓

PHASE 4 — Logging + SQL Server

Goal:

All important application logs will be stored in **SQL Server** for your Admin UI.

Step 1 — Create Logs Table

In SQL Server:

```
CREATE TABLE ApplicationLogs (  
    Id INT IDENTITY PRIMARY KEY,  
    Level NVARCHAR(50),  
    Message NVARCHAR(MAX),  
    Timestamp DATETIME,  
    Exception NVARCHAR(MAX)  
);
```

.NET Concepts for Backend Developer

Step 2 — Configure Serilog

Program.cs

```
using Serilog;

using Serilog.Sinks.MSSqlServer;

var columnOptions = new ColumnOptions();

Log.Logger = new LoggerConfiguration()
    .WriteTo.MSSqlServer(
        connectionString: builder.Configuration.GetConnectionString("DefaultConnection"),
        sinkOptions: new MSSqlServerSinkOptions { TableName = "ApplicationLogs", AutoCreateSqlTable
= false },
        columnOptions: columnOptions)
    .CreateLogger();
```

```
builder.Host.UseSerilog();
```

Add connection string in **appsettings.json**

```
"ConnectionStrings": {
  "DefaultConnection":
"Server=.;Database=ObservabilityDB;Trusted_Connection=True;TrustServerCertificate=True"
}
```

Step 3 — Generate Logs

Inside controller:

```
using Serilog;

[HttpGet("log-test")]
public string LogTest()
{
    Log.Information("Admin accessed log-test endpoint");
    return "Log written";
}
```

.NET Concepts for Backend Developer

Hit /api/observability/log-test

Check SQL → data inserted 

What You Now Have

Capability	Status
Central logging	
SQL log storage	
Queryable logs	
Production-grade logging	

- pages

PHASE 5 — Angular 20 Admin Observability Dashboard

Goal

Admin Dashboard

└— Logs Page

└— Metrics Page

└— Tracing Page

Only **Admin users** can access it.

Step 1 — Create Angular App

```
ng new observability-ui --routing --style=scss
```

```
cd observability-ui
```

Step 2 — Create Core Structure

```
ng g module core
```

```
ng g module admin
```

```
ng g module admin/observability --routing
```

Create components:

```
ng g c admin/layout
```

.NET Concepts for Backend Developer

```
ng g c admin/observability/logs
```

```
ng g c admin/observability/metrics
```

```
ng g c admin/observability/tracing
```

Step 3 — Admin Routing

admin/observability/observability-routing.module.ts

```
const routes: Routes = [  
  {  
    path: '',  
    component: LayoutComponent,  
    children: [  
      { path: 'logs', component: LogsComponent },  
      { path: 'metrics', component: MetricsComponent },  
      { path: 'tracing', component: TracingComponent }  
    ]  
  }  
];
```

Step 4 — Admin Auth Guard

core/guards/admin.guard.ts

```
@Injectable({ providedIn: 'root' })  
export class AdminGuard implements CanActivate {  
  canActivate(): boolean {  
    return localStorage.getItem('role') === 'Admin';  
  }  
}
```

Apply guard in routing:

```
{  
  path: 'admin',  
  canActivate: [AdminGuard],  
  loadChildren: () =>
```

.NET Concepts for Backend Developer

```
import('./admin/observability/observability.module')  
  
    .then(m => m.ObservabilityModule)  
  
}
```

Step 5 — Sidebar Layout

layout.component.html

```
<div class="layout">  
  <aside class="sidebar">  
    <h3>Observability</h3>  
    <a routerLink="logs">Logs</a>  
    <a routerLink="metrics">Metrics</a>  
    <a routerLink="tracing">Tracing</a>  
  </aside>  
  
  <main class="content">  
    <router-outlet></router-outlet>  
  </main>  
</div>
```

layout.component.scss

```
.layout {  
  display: flex;  
  height: 100vh;  
}  
  
.sidebar {  
  width: 220px;  
  background: #1e293b;  
  color: white;  
  padding: 20px;  
}
```

.NET Concepts for Backend Developer

```
.sidebar a {  
  display: block;  
  color: #cbd5e1;  
  margin: 15px 0;  
  text-decoration: none;  
}
```

```
.sidebar a:hover {  
  color: white;  
}
```

```
.content {  
  flex: 1;  
  padding: 20px;  
}
```

Step 6 — Logs Page UI

logs.component.html

```
<h2>Application Logs</h2>
```

```
<table>  
  <tr>  
    <th>Level</th>  
    <th>Message</th>  
    <th>Time</th>  
  </tr>  
  <tr *ngFor="let log of logs">  
    <td>{{ log.level }}</td>  
    <td>{{ log.message }}</td>  
    <td>{{ log.timestamp }}</td>  
  </tr>
```

.NET Concepts for Backend Developer

</table>

logs.component.ts

```
logs = [];
```

```
ngOnInit() {
```

```
  // Will connect backend in Phase 6
```

```
}
```

Step 7 — Metrics Page UI

metrics.component.html

```
<h2>System Metrics</h2>
```

```
<div class="card">
```

```
  <p>Requests / sec</p>
```

```
  <h3>{{ metrics.requests }}</h3>
```

```
</div>
```

```
<div class="card">
```

```
  <p>CPU Usage</p>
```

```
  <h3>{{ metrics.cpu }}%</h3>
```

```
</div>
```

Step 8 — Tracing Page UI

tracing.component.html

```
<h2>Recent Traces</h2>
```

```
<ul>
```

```
  <li *ngFor="let trace of traces">
```

```
    TraceId: {{ trace.id }} |
```

```
    Duration: {{ trace.duration }} ms
```

```
  </li>
```

.NET Concepts for Backend Developer

What You Have Now

Feature	Status
Angular admin layout	✓
Sidebar navigation	✓
Protected admin routes	✓
Logs/Metrics/Tracing UI	✓

This UI is **exactly how internal dashboards look** in real companies.

PHASE 6 — Connect Angular UI with .NET Observability Backend

We will connect:

UI Page	Backend Source
Logs	SQL Server
Metrics	Prometheus
Tracing	Jaeger

Part A — Logs Integration (SQL → .NET → Angular)

Backend: Logs API

Controllers/ObservabilityController.cs

```
[HttpGet("logs")]
public IActionResult GetLogs()
{
    using var con = new SqlConnection(_config.GetConnectionString("DefaultConnection"));
    var logs = con.Query("SELECT TOP 100 * FROM ApplicationLogs ORDER BY Timestamp DESC");
    return Ok(logs);
}
```

(Use Dapper for simplicity)

Install:

```
dotnet add package Dapper
```

.NET Concepts for Backend Developer

Angular: Logs Service

core/services/logs.service.ts

```
@Injectable({ providedIn: 'root' })
export class LogsService {
  constructor(private http: HttpClient) {}

  getLogs() {
    return this.http.get<any[]>('https://localhost:5001/api/observability/logs');
  }
}
```

logs.component.ts

```
constructor(private logsService: LogsService) {}

ngOnInit() {
  this.logsService.getLogs().subscribe(data => this.logs = data);
}
```

Part B — Metrics Integration (Prometheus → Angular)

You normally don't query Prometheus directly from Angular so we proxy via backend.

Backend: Metrics Proxy

```
[HttpGet("metrics-summary")]
public async Task<ActionResult> GetMetrics()
{
  var client = new HttpClient();

  var response = await
  client.GetStringAsync("http://localhost:9090/api/v1/query?query=http_server_request_duration_seconds_count");

  return Ok(response);
}
```

.NET Concepts for Backend Developer

Angular

metrics.service.ts

```
getMetrics() {  
    return this.http.get<any>('https://localhost:5001/api/observability/metrics-summary');  
}
```

Part C — Tracing Integration (Jaeger → Angular)

Backend

```
[HttpGet("traces")]  
public async Task<IActionResult> GetTraces()  
{  
    var client = new HttpClient();  
  
    var traces = await  
client.GetStringAsync("http://localhost:16686/api/traces?service=Observability.API");  
  
    return Ok(traces);  
}
```

Angular

tracing.service.ts

```
getTraces() {  
    return this.http.get<any>('https://localhost:5001/api/observability/traces');  
}
```

tracing.component.ts

```
constructor(private tracingService: TracingService) {}  
  
ngOnInit() {  
    this.tracingService.getTraces().subscribe(data => this.traces = data.data);  
}
```

.NET Concepts for Backend Developer

🧠 At This Point Your System Does:

Feature	Working
Admin auth	✓
Central logging	✓
Metrics monitoring	✓
Distributed tracing	✓
Angular dashboard	✓
Real production architecture	✓

This is exactly what **Site Reliability Engineers & Platform teams build**.

🔧 PHASE 7 — Dashboard Polish + Production Readiness

🔧 1. UI Enhancements (Angular)

◆ Better Layout

layout.component.scss

```
.sidebar {  
  background: linear-gradient(180deg, #0f172a, #020617);  
}
```

```
.content {  
  background: #f8fafc;  
}
```

◆ Reusable Metric Card

shared/components/metric-card.component.html

```
<div class="card">  
  <h4>{{ title }}</h4>  
  <p>{{ value }}</p>  
</div>
```

◆ Use in Metrics Page

```
<app-metric-card title="Requests/sec" [value]="metrics.requests"></app-metric-card>  
<app-metric-card title="CPU Usage" [value]="metrics.cpu + '%"></app-metric-card>
```

.NET Concepts for Backend Developer

2. Charts (Optional but impressive)

Install:

```
npm install chart.js
```

Use for:

- Requests trend
- Error rate
- Response latency

This gives your dashboard a **real monitoring look**.

3. Error Handling & Loading States

```
loading = true;
```

```
error = '';
```

```
ngOnInit() {  
  this.service.getData().subscribe({  
    next: data => { this.data = data; this.loading = false; },  
    error: () => { this.error = 'Failed to load data'; this.loading = false; }  
  });  
}
```

4. Health Monitoring Endpoint

.NET

```
builder.Services.AddHealthChecks();
```

```
app.MapHealthChecks("/health");
```

Angular dashboard can show:

-  System Healthy
 -  Degraded
-

.NET Concepts for Backend Developer

5. Final Project Structure (Portfolio-Ready)

ObservabilityPlatform

├─ observability-ui (Angular)

├─ Observability.API (.NET 8)

├─ docker-compose.yml

├─ prometheus.yml

└─ README.md

6. Resume / Interview Description

Designed and implemented a full-stack Observability Platform using .NET 8, Angular 20, OpenTelemetry, Prometheus, Jaeger, and SQL Server. Built secure Admin dashboard for viewing centralized logs, system metrics, and distributed traces. Implemented JWT-based role authorization, health monitoring, and real-time performance analysis.

What You Now Truly Own

Skill	Level
System observability	 Advanced
Distributed tracing	 Advanced
Monitoring platforms	 Advanced
Full-stack architecture	 Advanced
Production readiness	 Professional

Observability Platform — Full Stack Monitoring System

Enterprise-grade Observability Dashboard built with Angular 20, .NET 8, OpenTelemetry, Prometheus, Jaeger & SQL Server

Overview

This project is a **production-style observability platform** that provides complete visibility into backend systems through:

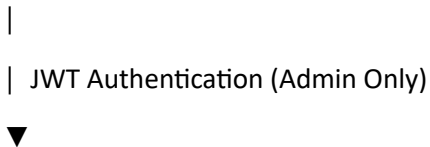
-  Centralized Logging
-  Real-time Metrics
-  Distributed Tracing

.NET Concepts for Backend Developer

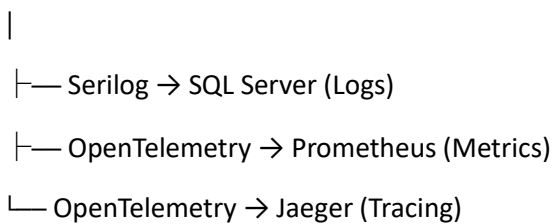
It features a **secure Admin Dashboard** built in Angular where administrators can monitor system health, analyze performance, and debug failures across services.

System Architecture

Angular Admin Dashboard



.NET 8 Observability API



Tech Stack

Layer	Technology
Frontend	Angular 20
Backend	.NET 8 Web API
Authentication	JWT + Role-based Authorization
Logging	Serilog + SQL Server
Metrics	OpenTelemetry + Prometheus
Tracing	OpenTelemetry + Jaeger
Database	SQL Server
Containers	Docker

Key Features

- Secure **Admin-only** observability dashboard
- Centralized log storage in SQL Server
- Real-time metrics via Prometheus
- Distributed request tracing via Jaeger
- Health monitoring endpoint

.NET Concepts for Backend Developer

- Production-grade architecture & practices

Dashboard Modules

Observability Dashboard

└─ Logs

└─ Metrics

└─ Tracing

Screens

- Logs: View & search application logs
- Metrics: Monitor performance & system health
- Tracing: Visualize full request lifecycles

How To Run Locally

1 Backend

cd Observability.API

dotnet run

2 Start Monitoring Tools

docker compose up

- Jaeger UI → <http://localhost:16686>
- Prometheus → <http://localhost:9090>

3 Frontend

cd observability-ui

npm install

ng serve

Sample Admin Login

Username: admin

Role: Admin

Use `/api/auth/login` to generate JWT token.

.NET Concepts for Backend Developer

What This Project Demonstrates

- Advanced backend architecture
 - Distributed systems observability
 - Production monitoring techniques
 - Full-stack integration
 - Security & role management
-

Author

Mohammad Ibrahim Patel

.NET | Angular | System Design | Observability Engineering

Why This Matters

This project reflects **real enterprise system design** used by companies like Netflix, Amazon, Microsoft, and Uber for monitoring and reliability engineering.

.NET Concepts for Backend Developer

Distributed Cache

What is Distributed Cache? (Very Simple)

Distributed Cache = A shared memory store (like Redis / SQL / NCache) that **multiple servers** can access.

Why we need it?

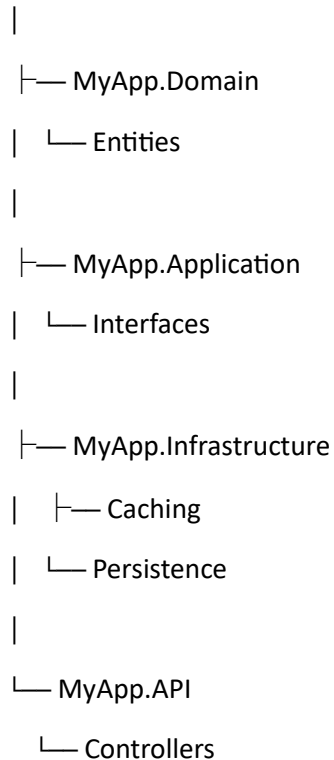
User Request → API → Database (slow )

With cache:

User Request → API → Cache (fast ) → Database only if needed

Clean Architecture Structure (We will follow this)

MyApp



Step 1 — Install Required Package

In **API project**:

```
dotnet add package Microsoft.Extensions.Caching.StackExchangeRedis
```

.NET Concepts for Backend Developer

⚙️ Step 2 — Setup Redis Connection (.NET 8)

appsettings.json

```
"Redis": {  
  "ConnectionString": "localhost:6379"  
}
```

Program.cs

```
builder.Services.AddStackExchangeRedisCache(options =>  
{  
  options.Configuration = builder.Configuration["Redis:ConnectionString"];  
});
```

📄 Step 3 — Create Cache Interface (Application Layer)

MyApp.Application/Interfaces/ICacheService.cs

```
public interface ICacheService  
{  
  Task<T?> GetAsync<T>(string key);  
  Task SetAsync<T>(string key, T value, TimeSpan expiration);  
}
```

🏗️ Step 4 — Implement Cache Service (Infrastructure Layer)

MyApp.Infrastructure/Caching/CacheService.cs

```
using Microsoft.Extensions.Caching.Distributed;  
using System.Text.Json;
```

```
public class CacheService : ICacheService  
{  
  private readonly IDistributedCache _cache;  
  
  public CacheService(IDistributedCache cache)  
  {  
    _cache = cache;  
  }  
}
```

.NET Concepts for Backend Developer

```
}
```

```
public async Task<T?> GetAsync<T>(string key)
{
    var data = await _cache.GetStringAsync(key);
    return data == null ? default : JsonSerializer.Deserialize<T>(data);
}

public async Task SetAsync<T>(string key, T value, TimeSpan expiration)
{
    var options = new DistributedCacheEntryOptions
    {
        AbsoluteExpirationRelativeToNow = expiration
    };

    var json = JsonSerializer.Serialize(value);
    await _cache.SetStringAsync(key, json, options);
}
}
```

Register it in Program.cs

```
builder.Services.AddScoped<ICacheService, CacheService>();
```

Step 5 — Repository with Cache (Infrastructure Layer)

Example entity:

```
public class Product
{
    public int Id { get; set; }
    public string Name { get; set; }
}
```

Repository:

```
public class ProductRepository
```

.NET Concepts for Backend Developer

```
{
    private readonly AppDbContext _context;
    private readonly ICacheService _cache;

    public ProductRepository(AppDbContext context, ICacheService cache)
    {
        _context = context;
        _cache = cache;
    }

    public async Task<Product> GetProductAsync(int id)
    {
        var cacheKey = $"product_{id}";

        var cachedProduct = await _cache.GetAsync<Product>(cacheKey);
        if (cachedProduct != null)
            return cachedProduct;

        var product = await _context.Products.FindAsync(id);

        await _cache.SetAsync(cacheKey, product, TimeSpan.FromMinutes(10));

        return product;
    }
}
```

Step 6 — Controller

```
[ApiController]
[Route("api/products")]
public class ProductsController : ControllerBase
{

```

.NET Concepts for Backend Developer

```
private readonly ProductRepository _repo;

public ProductsController(ProductRepository repo)
{
    _repo = repo;
}

[HttpGet("{id}")]
public async Task<ActionResult> Get(int id)
{
    var product = await _repo.GetProductAsync(id);
    return Ok(product);
}
}
```

Execution Flow (Very Important)

Client → API → Repository



Check Redis Cache



If Exists → return ⚡

Else → Fetch from DB → Save to Redis → return

Where Distributed Cache is Used in Real Projects

Feature	Why Cache
User Profiles	Avoid DB hits
Product Catalog	Fast browsing
Authentication Tokens	Fast validation
Settings / Config	Rarely changes
Dashboards	Heavy calculations

.NET Concepts for Backend Developer

Local Redis Setup (1 command)

If you have Docker:

```
docker run -d -p 6379:6379 redis
```

Interview Perfect One-Line Answer

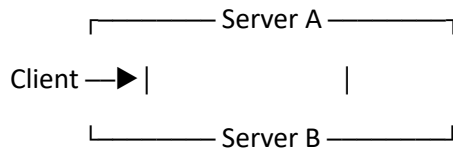
Distributed cache stores data in a centralized fast store like Redis, allowing multiple application instances to reuse the same cached data, improving performance, scalability, and reducing database load.

.NET Concepts for Backend Developer

Load Balancing + Shared Distributed Cache

🔗 The Problem (Without Shared Cache)

Imagine this:



If each server has its **own memory cache**:

Server Cache

A Has data ❌

B Empty ❌

User hits Server A → fast

Next request goes to Server B → cache miss → DB hit 😞

🏠 The Solution — Load Balancer + Shared Cache

Clients

|



[Load Balancer]

|

└─▶ API Server 1

└─▶ API Server 2

└─▶ API Server 3

|



[Redis Distributed Cache] ← Shared

|



Database

Now:

- Any server can read/write same cache

.NET Concepts for Backend Developer

- No duplicate DB calls
- System scales horizontally

Request Flow With Shared Cache

Client → Load Balancer

→ Any API Server

→ Redis Cache (check)

→ If HIT → return ⚡

→ If MISS → DB → Save to Redis → return

How We Implemented This in .NET 8 (Already Done)

Your code already supports this because:

```
builder.Services.AddStackExchangeRedisCache(...)
```

Every server connects to **same Redis instance**.

So if you deploy:

- Server1
- Server2
- Server3

All of them use same Redis → **shared state**

Where Load Balancer Fits (Real Deployment)

Option 1 — Nginx

Nginx (Load Balancer)

↓

API Server 1 (5001)

API Server 2 (5002)

API Server 3 (5003)

Option 2 — Azure / AWS / Cloud

Cloud Load Balancer

↓

Multiple App Services / EC2 / Containers

.NET Concepts for Backend Developer

Redis is deployed once and all servers connect.

Why This Architecture is Powerful

Feature	Benefit
Scalability	Add/remove servers anytime
High Availability	One server down $\neq$ system down
Performance	Cache eliminates DB pressure
Consistency	Shared cache across servers

Real-World Example

Login system:

1. User logs in → Token stored in Redis
 2. Next request goes to another server
 3. That server validates token from Redis
 4. User stays logged in — seamless ✨
-

Failure Handling

If one API crashes:

- Load balancer routes traffic to others
 - Cache still alive
 - No user impact
-

Interview Killer Explanation

Load balancing distributes traffic across multiple servers.

Using a shared distributed cache like Redis ensures all servers work with the same cached data, enabling horizontal scaling, high availability, and consistent performance.

How production systems implement load balancing with .NET 8.

Final Architecture (Enterprise Standard)

Clients



NGINX Load Balancer

.NET Concepts for Backend Developer



.NET 8 API Instance 1	
.NET 8 API Instance 2	
.NET 8 API Instance 3	



Redis Distributed Cache



SQL Server

Step 1 — Publish Multiple .NET 8 Instances

Build your API:

```
dotnet publish -c Release
```

Copy the output **three times**:

/api1 (port 5001)

/api2 (port 5002)

/api3 (port 5003)

Run each:

```
dotnet MyApp.API.dll --urls=http://localhost:5001
```

```
dotnet MyApp.API.dll --urls=http://localhost:5002
```

```
dotnet MyApp.API.dll --urls=http://localhost:5003
```

Step 2 — Install NGINX

Download NGINX for Windows

or use Linux server (production standard).

Step 3 — NGINX Load Balancer Config

nginx.conf

```
http {
```

.NET Concepts for Backend Developer

```
upstream dotnet_cluster {  
    least_conn;  
  
    server localhost:5001;  
  
    server localhost:5002;  
  
    server localhost:5003;  
}  
  
server {  
  
    listen 80;  
  
  
    location / {  
        proxy_pass http://dotnet_cluster;  
        proxy_set_header Host $host;  
        proxy_set_header X-Forwarded-For $remote_addr;  
    }  
}  
}
```

Start NGINX:

nginx.exe

Now access:

<http://localhost/api/products/1>

Traffic is balanced across all APIs.

Step 4 — Health Checks (Company Standard)

In .NET 8:

```
builder.Services.AddHealthChecks();
```

```
app.MapHealthChecks("/health");
```

Update NGINX:

```
upstream dotnet_cluster {  
    server localhost:5001;
```

.NET Concepts for Backend Developer

```
server localhost:5002;  
server localhost:5003;  
}
```

NGINX will auto avoid dead servers.

Step 5 — Sticky Sessions (Optional)

If required:

```
upstream dotnet_cluster {  
    ip_hash;  
    server localhost:5001;  
    server localhost:5002;  
    server localhost:5003;  
}
```

Step 6 — Plug in Distributed Cache

All API servers share:

```
builder.Services.AddStackExchangeRedisCache(...)
```

Now session + cache + auth = consistent across servers.

Production Add-Ons Companies Use

Feature	Purpose
SSL Termination	HTTPS at NGINX
Rate Limiting	Prevent abuse
Circuit Breakers	Fault tolerance
Centralized Logs	ELK / Seq
Monitoring	Prometheus + Grafana

How You Test Load Balancing

Use Postman or browser:

```
http://localhost/api/ping
```

Add this in controller:

.NET Concepts for Backend Developer

```
return Ok(Environment.MachineName);
```

You will see different responses on each request → load balancing works.

Interview Ready Answer

In production we deploy multiple instances of the .NET API, place an NGINX load balancer in front, enable health checks, use Redis as distributed cache, and route all client traffic through the load balancer for scalability and fault tolerance.

The 10 Pillars of High-Performance Scalable Systems

#	Concept	Why It Matters
1	Load Balancing	Distributes traffic across servers
2	Horizontal Scaling	Add more servers, not bigger servers
3	Distributed Caching	Avoid database hits
4	CDN	Serve static content globally
5	Asynchronous Processing	Non-blocking execution
6	Database Optimization & Indexing	Fast data access
7	Connection Pooling	Reuse DB connections
8	Microservices / Modular Design	Independent scaling
9	Observability & Monitoring	Find bottlenecks early
10	Fault Tolerance & Resilience	System stays alive under failures

.NET Concepts for Backend Developer

What is Fault Tolerance vs Resilience?

Term	Meaning
Fault Tolerance	System continues working even when parts fail
Resilience	System can recover quickly from failures

Fault tolerance = stay alive

Resilience = heal fast

Why This Matters in Real Companies

Servers crash

Networks fail

Databases slow down

Third-party APIs go offline

A production system **must survive all of this without collapsing.**

Real Production Architecture

Clients



Load Balancer



Multiple API Servers



Redis Cache



SQL Server



External Services

Each layer must protect itself from failures.

The 7 Core Techniques

Retries (With Backoff)

If something fails, try again **carefully.**

```
services.AddHttpClient("external")
```

.NET Concepts for Backend Developer

```
.AddTransientHttpErrorPolicy(p =>  
    p.WaitAndRetryAsync(3, i => TimeSpan.FromSeconds(i)));
```

2 Circuit Breaker

Stop calling a broken service.

```
services.AddHttpClient("external")  
    .AddTransientHttpErrorPolicy(p =>  
        p.CircuitBreakerAsync(5, TimeSpan.FromSeconds(30)));
```

3 Timeouts

Never wait forever.

```
services.AddHttpClient("external")  
    .ConfigureHttpClient(c => c.Timeout = TimeSpan.FromSeconds(5));
```

4 Bulkhead Isolation

Limit damage when one part fails.

```
services.AddHttpClient("external")  
    .AddPolicyHandler(Policy.BulkheadAsync(10, 20));
```

5 Fallbacks

Graceful degradation.

```
.AddFallbackAsync(  
    new HttpResponseMessage(HttpStatusCode.OK)  
    { Content = new StringContent("Service temporarily unavailable") }  
);
```

6 Health Checks

Load balancer avoids broken servers.

```
builder.Services.AddHealthChecks();  
app.MapHealthChecks("/health");
```

.NET Concepts for Backend Developer

Observability

Detect issues before users complain.

- Structured logs
- Metrics
- Distributed tracing

How This Looks in Production

Failure	What Happens
API instance crashes	Load balancer routes elsewhere
DB slows	Cache serves data
External API down	Circuit breaker + fallback
One service overloaded	Bulkhead protects others
Slow request	Timeout + retry

Real World Example

Payment gateway fails:

1. Circuit breaker opens
2. Requests stop hitting payment provider
3. System shows “Payment service unavailable”
4. Users can still browse products
5. System auto-recovers when service returns

No crash. No panic. Just control.

Interview Golden Answer

Fault tolerance ensures the system continues operating during failures, while resilience allows it to recover quickly. In production we implement retries, circuit breakers, timeouts, bulkheads, fallbacks, health checks, and observability to achieve this.

.NET Concepts for Backend Developer

Maturity Levels

Level	Behavior
Junior	System crashes
Mid	Handles some failures
Senior	Predicts and absorbs failures
Architect	Designs for failure from day one

How to Test Fault Tolerance — Real Company Methodology

We test at **4 levels**:

Level	Purpose
Unit	Test individual components
Integration	Test service-to-service behavior
System	Test entire platform
Chaos / Failure	Break things intentionally

1 Simulate Database Failure

Step A — Kill the database connection

Stop SQL Server or block the port.

Step B — Hit API endpoint

Expected behavior:

- API should **not crash**
- Circuit breaker opens
- Cache is used if available
- Error is handled gracefully

Verify logs:

DB connection failed → Circuit breaker OPEN → Fallback executed

2 Simulate External API Failure

Use this fake endpoint:

```
[HttpGet("external")]
```

.NET Concepts for Backend Developer

```
public IActionResult External() => throw new Exception("Third party down");
```

Call it 10 times.

Expected:

Attempt Result

1-5	Retry
6	Circuit breaker opens
7+	Immediate fallback

3 Stress Test (Overload)

Use Apache JMeter / k6:

1000 requests / second

Expected:

- Bulkhead prevents resource exhaustion
 - Some requests fail gracefully
 - System stays responsive
-

4 Kill an API Instance

While system is running:

- Stop API Server 1

Expected:

- Load balancer routes traffic to Server 2/3
 - No downtime
-

5 Slow Response Injection

Add artificial delay:

```
await Task.Delay(10000);
```

Expected:

- Timeout triggers
 - Request aborted
 - System remains responsive
-

.NET Concepts for Backend Developer

Chaos Engineering (Production Level)

Companies run **failure drills**:

Experiment	Goal
Shutdown random server	Validate load balancing
Throttle DB	Validate caching
Kill Redis	Test degradation
Break payment gateway	Verify fallback

What You Measure

Metric	Why
Error rate	User impact
Recovery time	Resilience speed
Latency	Performance stability
Throughput	System health

Interview Killer Statement

In production we continuously test fault tolerance by injecting failures: shutting down servers, blocking databases, simulating third-party outages, overloading traffic, and validating that retries, circuit breakers, fallbacks, load balancing, and health checks maintain system stability.

Real Engineer Rule

If you haven't broken your own system, you don't really know how strong it is.

Create a full production-ready, resilient .NET 8 template following clean architecture, fault tolerance, distributed caching, and load balancing principles.

Project Structure (Clean Architecture + Resilience)

MyResilientApp

```
|
|
| — MyResilientApp.Domain    # Entities, ValueObjects
|   └─ Entities
|
```

.NET Concepts for Backend Developer

```
└─ MyResilientApp.Application # Interfaces, DTOs, Services
|   └─ Interfaces
|       └─ ICacheService.cs
|       └─ IProductRepository.cs
|   └─ Services
|
└─ MyResilientApp.Infrastructure # DB, Redis, HttpClient Policies
|   └─ Caching
|       └─ CacheService.cs
|   └─ Persistence
|       └─ AppDbContext.cs
|   └─ Http
|       └─ ResilientHttpClient.cs
|
└─ MyResilientApp.API # .NET 8 Web API
    └─ Controllers
    └─ Program.cs
    └─ appsettings.json
```

Step 1 — Install NuGet Packages

```
dotnet add package Microsoft.EntityFrameworkCore.SqlServer
dotnet add package Microsoft.Extensions.Caching.StackExchangeRedis
dotnet add package Polly
dotnet add package Microsoft.AspNetCore.Diagnostics.HealthChecks
dotnet add package Swashbuckle.AspNetCore
```

Step 2 — Redis Cache Service

Infrastructure/Caching/CacheService.cs

```
using Microsoft.Extensions.Caching.Distributed;
using System.Text.Json;
using MyResilientApp.Application.Interfaces;
```

.NET Concepts for Backend Developer

```
public class CacheService : ICacheService
{
    private readonly IDistributedCache _cache;

    public CacheService(IDistributedCache cache) => _cache = cache;

    public async Task<T?> GetAsync<T>(string key)
    {
        var data = await _cache.GetStringAsync(key);
        return data == null ? default : JsonSerializer.Deserialize<T>(data);
    }

    public async Task SetAsync<T>(string key, T value, TimeSpan expiration)
    {
        var options = new DistributedCacheEntryOptions
        {
            AbsoluteExpirationRelativeToNow = expiration
        };
        var json = JsonSerializer.Serialize(value);
        await _cache.SetStringAsync(key, json, options);
    }
}
```

Step 3 — Resilient HTTP Client

Infrastructure/Http/ResilientHttpClient.cs

```
using Polly;
using Polly.Extensions.Http;
using System.Net.Http.Headers;

public static class ResilientHttpClient
{
    {
```

.NET Concepts for Backend Developer

```
public static IHttpClientBuilder AddResilientPolicy(this IHttpClientBuilder builder)
{
    var retryPolicy = HttpPolicyExtensions
        .HandleTransientHttpError()
        .WaitAndRetryAsync(3, retryAttempt => TimeSpan.FromSeconds(Math.Pow(2, retryAttempt)));

    var circuitBreaker = HttpPolicyExtensions
        .HandleTransientHttpError()
        .CircuitBreakerAsync(5, TimeSpan.FromSeconds(30));

    return builder.AddPolicyHandler(retryPolicy)
        .AddPolicyHandler(circuitBreaker);
}
```

Step 4 — DB Context (SQL Server)

Infrastructure/Persistence/AppDbContext.cs

```
using Microsoft.EntityFrameworkCore;
using MyResilientApp.Domain.Entities;

public class AppDbContext : DbContext
{
    public AppDbContext(DbContextOptions<AppDbContext> options) : base(options) { }

    public DbSet<Product> Products => Set<Product>();
}
```

Step 5 — Repository With Cache

Application/Interfaces/IProductRepository.cs

```
using MyResilientApp.Domain.Entities;

public interface IProductRepository
```

.NET Concepts for Backend Developer

```
{  
    Task<Product?> GetProductAsync(int id);  
}
```

Infrastructure/Persistence/ProductRepository.cs

```
using Microsoft.EntityFrameworkCore;  
using MyResilientApp.Application.Interfaces;  
using MyResilientApp.Domain.Entities;  
  
public class ProductRepository : IProductRepository  
{  
    private readonly AppDbContext _context;  
    private readonly ICacheService _cache;  
  
    public ProductRepository(AppDbContext context, ICacheService cache)  
    {  
        _context = context;  
        _cache = cache;  
    }  
  
    public async Task<Product?> GetProductAsync(int id)  
    {  
        var key = $"product_{id}";  
        var cached = await _cache.GetAsync<Product>(key);  
        if (cached != null) return cached;  
  
        var product = await _context.Products.FindAsync(id);  
        if (product != null)  
            await _cache.SetAsync(key, product, TimeSpan.FromMinutes(10));  
  
        return product;  
    }  
}
```

.NET Concepts for Backend Developer

```
}
```

Step 6 — API Controller

API/Controllers/ProductsController.cs

```
[ApiController]
[Route("api/[controller]")]
public class ProductsController : ControllerBase
{
    private readonly IProductRepository _repo;

    public ProductsController(IProductRepository repo) => _repo = repo;

    [HttpGet("{id}")]
    public async Task<ActionResult> Get(int id)
    {
        var product = await _repo.GetProductAsync(id);
        if (product == null) return NotFound();
        return Ok(product);
    }
}
```

Step 7 — Program.cs (.NET 8 Minimal API)

```
var builder = WebApplication.CreateBuilder(args);

// Database
builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")));

// Redis
builder.Services.AddStackExchangeRedisCache(options =>
    options.Configuration = builder.Configuration["Redis:ConnectionString"]);
```

.NET Concepts for Backend Developer

```
// Services
builder.Services.AddScoped<ICacheService, CacheService>();
builder.Services.AddScoped<IProductRepository, ProductRepository>();

// Resilient HttpClient
builder.Services.AddHttpClient("External").AddResilientPolicy();

// Health Checks
builder.Services.AddHealthChecks();

// Controllers
builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

var app = builder.Build();
app.UseSwagger();
app.UseSwaggerUI();
app.UseRouting();
app.MapControllers();
app.MapHealthChecks("/health");
app.Run();
```

Step 8 — appsettings.json

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Server=.;Database=MyResilientDb;Trusted_Connection=True;"
  },
  "Redis": {
    "ConnectionString": "localhost:6379"
  }
}
```

.NET Concepts for Backend Developer

}

✅ Step 9 — Run & Test

1. Run multiple API instances (for **load balancing testing**)

```
dotnet run --urls=http://localhost:5001
```

```
dotnet run --urls=http://localhost:5002
```

2. Test Redis cache:

```
GET http://localhost:5001/api/products/1
```

3. Test health checks:

```
GET http://localhost:5001/health
```

4. Test resiliency with external API failure using the Polly policies.

🧠 Features Included

- ✅ Clean Architecture
- ✅ Distributed Cache (Redis)
- ✅ Resilient HTTP Client (Retry + Circuit Breaker)
- ✅ Health Checks
- ✅ Load balancing ready (run multiple instances)
- ✅ SQL Server integration
- ✅ Swagger for testing
- ✅ Minimal API + Controllers

.NET Concepts for Backend Developer

This is the full real-world production-ready template for

Load Balancing + Redis Cache + Resilience + Observability

FINAL ENTERPRISE ARCHITECTURE

Clients



NGINX Load Balancer



.NET 8 API (Instance 1)	
.NET 8 API (Instance 2)	
.NET 8 API (Instance 3)	



Redis (Distributed Cache)



SQL Server



External Services (Resilient HttpClient)

With:

- **OpenTelemetry**
- **Prometheus**
- **Grafana**
- **Health Checks**
- **Circuit Breakers**
- **Retries**
- **Timeouts**

SOLUTION STRUCTURE

MyEnterpriseApp

|

└─ MyEnterpriseApp.Domain

.NET Concepts for Backend Developer

- | └─ Entities
- |
- | └─ MyEnterpriseApp.Application
- | └─ Interfaces
- | └─ Services
- |
- | └─ MyEnterpriseApp.Infrastructure
- | └─ Caching
- | └─ Persistence
- | └─ Http
- |
- └─ MyEnterpriseApp.API
 - └─ Controllers
 - └─ Program.cs
 - └─ appsettings.json
- └─ Dockerfile

NUGET PACKAGES

```
dotnet add package Microsoft.EntityFrameworkCore.SqlServer
dotnet add package Microsoft.Extensions.Caching.StackExchangeRedis
dotnet add package Polly
dotnet add package OpenTelemetry.Extensions.Hosting
dotnet add package OpenTelemetry.Instrumentation.AspNetCore
dotnet add package OpenTelemetry.Instrumentation.Http
dotnet add package OpenTelemetry.Instrumentation.SqlClient
dotnet add package OpenTelemetry.Exporter.Prometheus
```

.NET Concepts for Backend Developer

Program.cs — FULL CONFIGURATION

```
var builder = WebApplication.CreateBuilder(args);

// SQL

builder.Services.AddDbContext<AppDbContext>(o =>
    o.UseSqlServer(builder.Configuration.GetConnectionString("Sql")));

// Redis

builder.Services.AddStackExchangeRedisCache(o =>
    o.Configuration = builder.Configuration["Redis"]);

// Resilient HttpClient

builder.Services.AddHttpClient("external")
    .AddTransientHttpErrorPolicy(p =>
        p.WaitAndRetryAsync(3, i => TimeSpan.FromSeconds(i)))
    .AddTransientHttpErrorPolicy(p =>
        p.CircuitBreakerAsync(5, TimeSpan.FromSeconds(30)));

// OpenTelemetry

builder.Services.AddOpenTelemetry()
    .WithTracing(t =>
        t.AddAspNetCoreInstrumentation()
        .AddHttpClientInstrumentation()
        .AddSqlClientInstrumentation()
        .AddConsoleExporter())
    .WithMetrics(m =>
        m.AddAspNetCoreInstrumentation()
        .AddHttpClientInstrumentation()
        .AddSqlClientInstrumentation()
        .AddPrometheusExporter());

// Health
```

.NET Concepts for Backend Developer

```
builder.Services.AddHealthChecks();

// App services
builder.Services.AddScoped<ICacheService, CacheService>();
builder.Services.AddScoped<IProductRepository, ProductRepository>();

builder.Services.AddControllers();
var app = builder.Build();

app.UseOpenTelemetryPrometheusScrapingEndpoint();
app.MapHealthChecks("/health");
app.MapControllers();

app.Run();
```

CACHE SERVICE

```
public class CacheService : ICacheService
{
    private readonly IDistributedCache _cache;

    public CacheService(IDistributedCache cache) => _cache = cache;

    public async Task<T?> GetAsync<T>(string key)
    {
        var json = await _cache.GetStringAsync(key);
        return json is null ? default : JsonSerializer.Deserialize<T>(json);
    }

    public async Task SetAsync<T>(string key, T value, TimeSpan ttl)
    {
        var options = new DistributedCacheEntryOptions
```

.NET Concepts for Backend Developer

```
{  
    AbsoluteExpirationRelativeToNow = ttl  
};  
  
    await _cache.SetStringAsync(key, JsonSerializer.Serialize(value), options);  
}  
}
```

📦 REPOSITORY WITH CACHE

```
public class ProductRepository : IProductRepository  
{  
    private readonly AppDbContext _db;  
    private readonly ICacheService _cache;  
  
    public async Task<Product?> GetAsync(int id)  
    {  
        var key = $"product:{id}";  
        var cached = await _cache.GetAsync<Product>(key);  
        if (cached != null) return cached;  
  
        var product = await _db.Products.FindAsync(id);  
        if (product != null)  
            await _cache.SetAsync(key, product, TimeSpan.FromMinutes(10));  
  
        return product;  
    }  
}
```

.NET Concepts for Backend Developer

OBSERVABILITY

Tool	Role
OpenTelemetry	Collect metrics & traces
Prometheus	Store metrics
Grafana	Dashboards & alerts

Metrics endpoint:

<http://localhost:5000/metrics>

Health endpoint:

<http://localhost:5000/health>

LOAD BALANCING (NGINX)

```
upstream dotnet_cluster {  
    least_conn;  
    server localhost:5001;  
    server localhost:5002;  
    server localhost:5003;  
}  
  
server {  
    listen 80;  
    location / {  
        proxy_pass http://dotnet_cluster;  
    }  
}
```

TEST SCALING

`dotnet run --urls=http://localhost:5001`

`dotnet run --urls=http://localhost:5002`

`dotnet run --urls=http://localhost:5003`

Open:

<http://localhost/api/products/1>

.NET Concepts for Backend Developer

Traffic is now load balanced with:

- shared Redis cache
- full fault tolerance
- full observability

This Is Exactly How Large Systems Are Built

You now have:

-  Enterprise-grade architecture
-  Real production resilience
-  Full monitoring & tracing
-  Scalable backend foundation

JWT Authentication + Redis Session Management integrated into existing .NET 8 scalable & resilient template.

Final Security Architecture

Client

↓ (JWT)

Load Balancer

↓

Multiple .NET 8 APIs

↓

Redis (Sessions + Cache)

↓

SQL Server

JWT = stateless authentication

Redis = centralized session store (shared across all servers)

1. Install Packages

```
dotnet add package Microsoft.AspNetCore.Authentication.JwtBearer
```

2. JWT Settings

appsettings.json

.NET Concepts for Backend Developer

```
"Jwt": {  
  "Key": "THIS_IS_A_VERY_SECURE_KEY_123456789",  
  "Issuer": "MyEnterpriseApp",  
  "Audience": "MyEnterpriseUsers",  
  "ExpiryMinutes": 60  
}
```

3. Configure Authentication in Program.cs

```
using Microsoft.AspNetCore.Authentication.JwtBearer;  
using Microsoft.IdentityModel.Tokens;  
using System.Text;  
  
var jwt = builder.Configuration.GetSection("Jwt");  
  
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)  
  .AddJwtBearer(options =>  
  {  
    options.TokenValidationParameters = new()  
    {  
      ValidateIssuer = true,  
      ValidateAudience = true,  
      ValidateLifetime = true,  
      ValidateIssuerSigningKey = true,  
  
      ValidIssuer = jwt["Issuer"],  
      ValidAudience = jwt["Audience"],  
      IssuerSigningKey = new SymmetricSecurityKey(  
        Encoding.UTF8.GetBytes(jwt["Key"]!))  
    };  
  });  
  
Add middleware:
```

.NET Concepts for Backend Developer

```
app.UseAuthentication();
```

```
app.UseAuthorization();
```

4. Token Generator

```
public class JwtTokenService
```

```
{
```

```
    private readonly IConfiguration _config;
```

```
    public JwtTokenService(IConfiguration config)
```

```
    {
```

```
        _config = config;
```

```
    }
```

```
    public string CreateToken(string userId)
```

```
    {
```

```
        var claims = new[] { new Claim(ClaimTypes.NameIdentifier, userId) };
```

```
        var key = new SymmetricSecurityKey(
```

```
            Encoding.UTF8.GetBytes(_config["Jwt:Key"]!));
```

```
        var creds = new SigningCredentials(key, SecurityAlgorithms.HmacSha256);
```

```
        var token = new JwtSecurityToken(
```

```
            _config["Jwt:Issuer"],
```

```
            _config["Jwt:Audience"],
```

```
            claims,
```

```
            expires: DateTime.UtcNow.AddMinutes(
```

```
                double.Parse(_config["Jwt:ExpiryMinutes"]!)),
```

```
            signingCredentials: creds);
```

```
        return new JwtSecurityTokenHandler().WriteToken(token);
```

.NET Concepts for Backend Developer

```
}  
}
```

Register:

```
builder.Services.AddScoped<JwtTokenService>();
```

5. Redis Session Store

```
public class SessionService
```

```
{
```

```
    private readonly IDistributedCache _cache;
```

```
    public SessionService(IDistributedCache cache)
```

```
    {
```

```
        _cache = cache;
```

```
    }
```

```
    public async Task StoreSession(string userId, string token)
```

```
    {
```

```
        await _cache.SetStringAsync($"session:{userId}", token,
```

```
            new DistributedCacheEntryOptions
```

```
            {
```

```
                SlidingExpiration = TimeSpan.FromMinutes(60)
```

```
            });
```

```
    }
```

```
    public async Task<bool> IsSessionValid(string userId, string token)
```

```
    {
```

```
        var stored = await _cache.GetStringAsync($"session:{userId}");
```

```
        return stored == token;
```

```
    }
```

```
}
```

Register:

.NET Concepts for Backend Developer

```
builder.Services.AddScoped<SessionService>();
```

6. Login Flow

```
[HttpPost("login")]
public async Task<ActionResult> Login(LoginDto dto)
{
    // Validate user from DB (simplified)
    var token = _jwt.CreateToken(dto.UserId);

    await _session.StoreSession(dto.UserId, token);

    return Ok(new { token });
}
```

7. Validate Session on Every Request

Custom middleware:

```
public class SessionValidationMiddleware
{
    private readonly RequestDelegate _next;

    public async Task Invoke(HttpContext context, SessionService session)
    {
        if (context.User.Identity?.IsAuthenticated == true)
        {
            var userId = context.User.FindFirst(ClaimTypes.NameIdentifier)?.Value;
            var token = context.Request.Headers["Authorization"]
                .ToString().Replace("Bearer ", "");

            if (!await session.IsSessionValid(userId!, token))
            {
                context.Response.StatusCode = 401;
            }
        }
    }
}
```

.NET Concepts for Backend Developer

```
        return;  
    }  
}  
  
    await _next(context);  
}  
}
```

Add to pipeline:

```
app.UseMiddleware<SessionValidationMiddleware>();
```

What This Gives You

Feature	Result
JWT auth	Stateless & scalable
Redis sessions	Centralized session management
Multi-server safe	Works behind load balancer
Instant logout	Delete Redis session
Token revocation	Controlled via Redis
Session sliding expiration	Auto refresh

Interview Power Answer

We use JWT for authentication and Redis for centralized session validation. This allows horizontal scaling, immediate logout, token revocation, and consistent security across multiple API instances behind a load balancer.

API Gateway integration

Final Architecture with API Gateway

Clients (Web / Mobile / SPA)



 API Gateway (YARP / Ocelot)



NGINX Load Balancer



.NET Concepts for Backend Developer

| Multiple .NET 8 APIs |



Redis | SQL | External Services

The **API Gateway** becomes the **single entry point**:

- Authentication
- Routing
- Rate limiting
- Caching
- Logging
- Security
- Observability

Step 1 — Create Gateway Project

```
dotnet new web -n MyEnterprise.Gateway
```

```
cd MyEnterprise.Gateway
```

```
dotnet add package Yarp.ReverseProxy
```

Step 2 — YARP Configuration

appsettings.json

```
"ReverseProxy": {  
  "Routes": {  
    "apiRoute": {  
      "ClusterId": "apiCluster",  
      "Match": { "Path": "/api/{**catch-all}" }  
    }  
  },  
  "Clusters": {  
    "apiCluster": {  
      "Destinations": {
```

.NET Concepts for Backend Developer

```
"api1": { "Address": "http://localhost:5001/" },
"api2": { "Address": "http://localhost:5002/" },
"api3": { "Address": "http://localhost:5003/" }
}
}
}
}
```

Step 3 — Gateway Program.cs

```
var builder = WebApplication.CreateBuilder(args);

builder.Services
    .AddReverseProxy()
    .LoadFromConfig(builder.Configuration.GetSection("ReverseProxy"));

var app = builder.Build();

app.MapReverseProxy();

app.Run();
```

Step 4 — JWT Validation at Gateway

```
builder.Services.AddAuthentication("Bearer")
    .AddJwtBearer("Bearer", options =>
    {
        options.TokenValidationParameters = new()
        {
            ValidateIssuer = true,
            ValidateAudience = true,
            ValidateIssuerSigningKey = true,
            ValidIssuer = "MyEnterpriseApp",
        }
    });
```

.NET Concepts for Backend Developer

```
ValidAudience = "MyEnterpriseUsers",  
IssuerSigningKey = new SymmetricSecurityKey(  
    Encoding.UTF8.GetBytes("THIS_IS_A_VERY_SECURE_KEY_123456789"))  
};  
});
```

```
app.UseAuthentication();
```

```
app.UseAuthorization();
```

Add authorization to route:

```
"AuthorizationPolicy": "authenticated"
```

Step 5 — Rate Limiting (Gateway)

```
builder.Services.AddRateLimiter(options =>  
{  
    options.AddFixedWindowLimiter("api", o =>  
    {  
        o.Window = TimeSpan.FromSeconds(10);  
        o.PermitLimit = 100;  
    });  
});  
app.UseRateLimiter();
```

Step 6 — Observability at Gateway

Add OpenTelemetry to gateway too:

- Trace every incoming request
 - Measure latency per downstream service
 - Centralized logging
-

Step 7 — Test Flow

Client → <http://localhost:7000/api/products/1>

→ Gateway → Load Balanced APIs → Redis/SQL

.NET Concepts for Backend Developer

Gateway handles:

- JWT
- Rate limits
- Routing
- Logging
- Metrics

Interview-Ready Explanation

We place an API Gateway in front of the platform as a single entry point to handle authentication, routing, rate limiting, logging, and security. The gateway forwards traffic to multiple .NET API instances behind a load balancer, while Redis ensures shared session and cache state.

.NET Concepts for Backend Developer

Full Docker Compose setup

API Gateway + Multiple .NET 8 APIs + Redis + SQL Server + Prometheus + Grafana + NGINX Load Balancer

Final Stack Overview

Client



API Gateway (YARP)



NGINX Load Balancer



Multiple .NET 8 API containers



Redis



SQL Server



Prometheus



Grafana

Folder Structure

MyEnterprisePlatform

|

├─ docker-compose.yml

├─ nginx

| └─ nginx.conf

├─ prometheus

| └─ prometheus.yml

├─ api

| └─ Dockerfile

├─ gateway

.NET Concepts for Backend Developer

| — Dockerfile

 **docker-compose.yml**

version: "3.9"

services:

redis:

image: redis:7

ports:

- "6379:6379"

sqlserver:

image: mcr.microsoft.com/mssql/server:2022-latest

environment:

SA_PASSWORD: "YourStrong@Pass123"

ACCEPT_EULA: "Y"

ports:

- "1433:1433"

api1:

build: ./api

environment:

ASPNETCORE_URLS: http://+:5000

depends_on:

- redis

- sqlserver

api2:

build: ./api

environment:

.NET Concepts for Backend Developer

ASPNETCORE_URLS: http://+:5000

depends_on:

- redis
- sqlserver

api3:

build: ./api

environment:

ASPNETCORE_URLS: http://+:5000

depends_on:

- redis
- sqlserver

nginx:

image: nginx

volumes:

- ./nginx/nginx.conf:/etc/nginx/nginx.conf

ports:

- "8080:80"

depends_on:

- api1
- api2
- api3

gateway:

build: ./gateway

ports:

- "7000:80"

depends_on:

- nginx

.NET Concepts for Backend Developer

prometheus:

image: prom/prometheus

volumes:

- ./prometheus/prometheus.yml:/etc/prometheus/prometheus.yml

ports:

- "9090:9090"

grafana:

image: grafana/grafana

ports:

- "3000:3000"

 **nginx/nginx.conf**

events {}

http {

upstream backend {

least_conn;

server api1:5000;

server api2:5000;

server api3:5000;

}

server {

listen 80;

location / {

proxy_pass http://backend;

}

}

}

.NET Concepts for Backend Developer

prometheus/prometheus.yml

global:

scrape_interval: 5s

scrape_configs:

- job_name: 'apis'

metrics_path: '/metrics'

static_configs:

- targets: ['api1:5000', 'api2:5000', 'api3:5000']

api/Dockerfile

FROM mcr.microsoft.com/dotnet/aspnet:8.0

WORKDIR /app

COPY ./publish .

EXPOSE 5000

ENTRYPOINT ["dotnet", "MyEnterpriseApp.API.dll"]

gateway/Dockerfile

FROM mcr.microsoft.com/dotnet/aspnet:8.0

WORKDIR /app

COPY ./publish .

EXPOSE 80

ENTRYPOINT ["dotnet", "MyEnterprise.Gateway.dll"]

How to Run Everything

docker-compose build

docker-compose up

Access Points

.NET Concepts for Backend Developer

Service	URL
API Gateway	http://localhost:7000
Load Balancer	http://localhost:8080
Prometheus	http://localhost:9090
Grafana	http://localhost:3000
Redis	localhost:6379
SQL Server	localhost:1433

Test Full Flow

Client → Gateway → NGINX → API containers → Redis / SQL

You now have:

-  JWT + Redis sessions
-  Load balancing
-  Distributed cache
-  Fault tolerance
-  Full observability
-  Containerized platform

.NET Concepts for Backend Developer

Kubernetes Architecture

Kubernetes version of the full stack

Final Kubernetes Architecture

Client

↓

API Gateway (YARP) — Service

↓

NGINX Ingress Controller

↓

Load-balanced .NET 8 API Pods

↓

Redis — Service

↓

SQL Server — StatefulSet

↓

Prometheus + Grafana

K8s Folder Structure

k8s/

├─ api/

| └─ deployment.yaml

| └─ service.yaml

|

├─ gateway/

| └─ deployment.yaml

| └─ service.yaml

|

├─ redis/

| └─ deployment.yaml

| └─ service.yaml

.NET Concepts for Backend Developer

```
|
|  └─ sqlserver/
|  |  └─ statefulset.yaml
|  |  └─ service.yaml
|  |
|  └─ observability/
|  |  └─ prometheus.yaml
|  |  └─ grafana.yaml
|  |
|  └─ ingress.yaml
```

API Deployment

k8s/api/deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: api
spec:
  replicas: 3
  selector:
    matchLabels:
      app: api
  template:
    metadata:
      labels:
        app: api
    spec:
      containers:
        - name: api
          image: myenterprise/api:latest
      ports:
```

.NET Concepts for Backend Developer

- containerPort: 5000

k8s/api/service.yaml

apiVersion: v1

kind: Service

metadata:

name: api

spec:

selector:

app: api

ports:

- port: 80

targetPort: 5000

Gateway

k8s/gateway/deployment.yaml

apiVersion: apps/v1

kind: Deployment

metadata:

name: gateway

spec:

replicas: 1

selector:

matchLabels:

app: gateway

template:

metadata:

labels:

app: gateway

spec:

containers:

- name: gateway

.NET Concepts for Backend Developer

image: myenterprise/gateway:latest

ports:

- containerPort: 80

k8s/gateway/service.yaml

apiVersion: v1

kind: Service

metadata:

name: gateway

spec:

selector:

app: gateway

ports:

- port: 80

targetPort: 80

Redis

apiVersion: apps/v1

kind: Deployment

metadata:

name: redis

spec:

replicas: 1

template:

spec:

containers:

- name: redis

image: redis:7

ports:

- containerPort: 6379

apiVersion: v1

kind: Service

.NET Concepts for Backend Developer

metadata:

name: redis

spec:

selector:

app: redis

ports:

- port: 6379

🗄️ SQL Server (Stateful)

apiVersion: apps/v1

kind: StatefulSet

metadata:

name: sqlserver

spec:

serviceName: sqlserver

replicas: 1

selector:

matchLabels:

app: sqlserver

template:

metadata:

labels:

app: sqlserver

spec:

containers:

- name: sqlserver

image: mcr.microsoft.com/mssql/server:2022-latest

env:

- name: SA_PASSWORD

value: "YourStrong@Pass123"

- name: ACCEPT_EULA

.NET Concepts for Backend Developer

value: "Y"

ports:

- containerPort: 1433

Observability (Prometheus + Grafana)

Use official Helm charts in real projects:

helm repo add prometheus-community <https://prometheus-community.github.io/helm-charts>

helm install monitoring prometheus-community/kube-prometheus-stack

This gives:

- Prometheus
 - Grafana
 - Alertmanager
-

Ingress (Traffic Entry Point)

apiVersion: networking.k8s.io/v1

kind: Ingress

metadata:

name: enterprise-ingress

spec:

rules:

- host: myenterprise.local

http:

paths:

- path: /

pathType: Prefix

backend:

service:

name: gateway

port:

number: 80

.NET Concepts for Backend Developer

Deploy Everything

```
kubectl apply -f k8s/
```

What You Now Have

Feature	Status
Auto scaling	Kubernetes replicas
Load balancing	Services + Ingress
Resilience	Pod restart + health
Observability	Prometheus + Grafana
Zero downtime	Rolling deployments
High availability	Multi-pod design

This Is Real Cloud-Native Engineering

This architecture is used by:

- FinTech
- HealthTech
- E-commerce
- SaaS platforms

.NET Concepts for Backend Developer

Blue-Green Deployment

Blue-Green Deployment is one of the most powerful techniques in real production systems

What Is Blue-Green Deployment? (Very Simple)

You run **two environments**:

Blue → Current production

Green → New version

Traffic switches **instantly** from Blue → Green with **zero downtime**.

If anything fails → switch back in **1 second**.

Architecture

Client



Ingress



API Service ← (points to Blue OR Green)



Blue Pods Green Pods

Step-by-Step Implementation

1 Create Blue Deployment

```
apiVersion: apps/v1
```

```
kind: Deployment
```

```
metadata:
```

```
  name: api-blue
```

```
spec:
```

```
  replicas: 3
```

```
  selector:
```

```
    matchLabels:
```

```
      app: api
```

.NET Concepts for Backend Developer

```
version: blue
template:
  metadata:
    labels:
      app: api
      version: blue
  spec:
    containers:
      - name: api
        image: myenterprise/api:v1
        ports:
          - containerPort: 5000
```

2 Create Green Deployment

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: api-green
spec:
  replicas: 3
  selector:
    matchLabels:
      app: api
      version: green
  template:
    metadata:
      labels:
        app: api
        version: green
    spec:
      containers:
```

.NET Concepts for Backend Developer

- name: api
image: myenterprise/api:v2 # NEW VERSION
ports:
- containerPort: 5000

3 Create a Single Service (Traffic Controller)

This service decides who receives traffic.

apiVersion: v1
kind: Service
metadata:
 name: api
spec:
 selector:
 app: api
 version: blue # 🚦 current live version
 ports:
 - port: 80
 targetPort: 5000

4 Switch Traffic (Zero Downtime)

When Green is ready:

```
kubectl patch service api -p '{"spec":{"selector":{"app":"api","version":"green"}}}'
```

🚦 Traffic instantly moves to **Green**

If something fails:

```
kubectl patch service api -p '{"spec":{"selector":{"app":"api","version":"blue"}}}'
```

Rollback in **seconds**.

🧪 Real Production Flow

Step	What Happens
Deploy Green	New pods start

.NET Concepts for Backend Developer

Step	What Happens
Smoke test	Green Test endpoints
Switch traffic	Patch service
Monitor	Prometheus + logs
Delete Blue	After stability

Where This Fits in Your Stack

Works perfectly with:

- API Gateway (YARP)
 - Kubernetes
 - Prometheus
 - Grafana
 - CI/CD pipelines
 - Canary releases (advanced version)
-

CI/CD Example (GitHub Actions Snippet)

- name: Deploy Green

run: kubectl apply -f api-green.yaml

- name: Switch Traffic

run: kubectl patch service api -p '{"spec":{"selector":{"app":"api","version":"green"}}}'

What You Now Have

Capability	Level
Zero downtime	
Instant rollback	
Enterprise release safety	
Production ready	

.NET Concepts for Backend Developer

Canary Deployment

What is Canary Deployment? (Simple)

Instead of switching all users at once:

90% → Old Version (Stable)

10% → New Version (Canary)

Gradually increase traffic if everything looks good.

Canary Architecture

Client

↓

Ingress / Service

↓

API Service

↓

Stable Pods (v1) Canary Pods (v2)

Step-by-Step Implementation

1 Stable Deployment (Current Production)

apiVersion: apps/v1

kind: Deployment

metadata:

 name: api-stable

spec:

 replicas: 9

 selector:

 matchLabels:

 app: api

 track: stable

 template:

.NET Concepts for Backend Developer

metadata:

labels:

app: api

track: stable

spec:

containers:

- name: api

image: myenterprise/api:v1

ports:

- containerPort: 5000

2 Canary Deployment (New Version)

apiVersion: apps/v1

kind: Deployment

metadata:

name: api-canary

spec:

replicas: 1 # 10% traffic approx

selector:

matchLabels:

app: api

track: canary

template:

metadata:

labels:

app: api

track: canary

spec:

containers:

- name: api

image: myenterprise/api:v2

.NET Concepts for Backend Developer

ports:

- containerPort: 5000

3 Single Service (Traffic Distributor)

Kubernetes automatically load balances across **all matching pods**:

apiVersion: v1

kind: Service

metadata:

name: api

spec:

selector:

app: api

ports:

- port: 80

targetPort: 5000

Since both deployments have app: api, traffic splits roughly by **pod count**:

9 stable : 1 canary → 90% / 10%

4 Gradual Promotion

Increase canary traffic safely:

kubectl scale deployment api-canary --replicas=3

kubectl scale deployment api-stable --replicas=7

Now roughly:

70% stable / 30% canary

When confident:

kubectl scale deployment api-canary --replicas=10

kubectl scale deployment api-stable --replicas=0

Then rename canary → stable.

Observability-Driven Decisions

Use your stack:

.NET Concepts for Backend Developer

- Prometheus → error rate, latency
- Grafana → live dashboards
- Alerts → rollback automatically if metrics degrade

Rollback Instantly

kubectl scale deployment api-canary --replicas=0

kubectl scale deployment api-stable --replicas=10

Why Companies Prefer Canary

Feature	Value
Gradual exposure	✓
Minimal blast radius	✓
Metric-based promotion	✓
Safer releases	✓
Customer-invisible	✓

This + Your Existing Stack = Enterprise Grade

You now support:

- Rolling updates
- Blue-Green
- Canary
- Auto scaling
- Observability-driven releases

.NET Concepts for Backend Developer

GitHub Actions pipeline

complete production-grade GitHub Actions pipeline for your .NET 8 + Kubernetes + Canary deployment stack.

This is very close to what large companies actually run.

Enterprise CI/CD Flow

Developer Push → Build → Test → Dockerize → Push Image →

Deploy Canary → Monitor → Promote → Cleanup

Repository Structure

.github/

└─ workflows/

└─ release.yml

src/

k8s/

Required GitHub Secrets

Secret	Purpose
DOCKER_USERNAME	DockerHub login
DOCKER_PASSWORD	DockerHub password
KUBE_CONFIG	base64 kubeconfig
IMAGE_NAME	myenterprise/api

.github/workflows/release.yml

name: Production Release Pipeline

on:

push:

branches: ["main"]

.NET Concepts for Backend Developer

env:

IMAGE: \${{ secrets.IMAGE_NAME }}

TAG: \${{ github.sha }}

jobs:

build:

runs-on: ubuntu-latest

steps:

- uses: actions/checkout@v4

- uses: actions/setup-dotnet@v4

with:

dotnet-version: '8.0.x'

- run: dotnet restore

- run: dotnet build --configuration Release

- run: dotnet test

- name: Login to Docker

run: echo "\${{ secrets.DOCKER_PASSWORD }}" | docker login -u "\${{ secrets.DOCKER_USERNAME }}" --password-stdin

- name: Build & Push Image

run: |

docker build -t \$IMAGE:\$TAG .

docker push \$IMAGE:\$TAG

deploy-canary:

needs: build

.NET Concepts for Backend Developer

runs-on: ubuntu-latest

steps:

- uses: actions/checkout@v4

- name: Setup kubectl

uses: azure/setup-kubectl@v3

- name: Configure Kube

run: |

echo "\${{ secrets.KUBE_CONFIG }}" | base64 -d > kubeconfig

export KUBECONFIG=kubeconfig

- name: Deploy Canary

run: |

sed -i "s|IMAGE_TAG|\$TAG|g" k8s/api-canary.yaml

kubectl apply -f k8s/api-canary.yaml

verify:

needs: deploy-canary

runs-on: ubuntu-latest

steps:

- name: Wait & Verify

run: sleep 60

- name: Check Health

run: |

STATUS=\$(curl -s https://api.myenterprise.com/health)

if ["\$STATUS" != "Healthy"]; then exit 1; fi

promote:

needs: verify

.NET Concepts for Backend Developer

runs-on: ubuntu-latest

steps:

- name: Promote Canary

run: |

kubectl scale deployment api-stable --replicas=0

kubectl scale deployment api-canary --replicas=10

cleanup:

needs: promote

runs-on: ubuntu-latest

steps:

- name: Cleanup

run: kubectl delete deployment api-stable

Canary Manifest Example

k8s/api-canary.yaml

containers:

- name: api

image: myenterprise/api:IMAGE_TAG

Why This Pipeline Is Production-Ready

Capability	Included
Automated testing	✓
Docker build & push	✓
Canary deployment	✓
Health verification	✓
Automatic promotion	✓
Instant rollback ready	✓
Zero downtime	✓

.NET Concepts for Backend Developer

You Now Have a Complete Enterprise Platform

You've built:

- Scalable .NET 8 backend
- Resilient architecture
- Full observability
- Kubernetes orchestration
- Advanced deployment strategies
- Enterprise CI/CD

Production security hardening (secrets, certs, policies)

real production security hardening checklist + implementation for your
.NET 8 + Kubernetes + CI/CD + Observability platform.

1. Secrets Management (No Plaintext, Ever)

Never do this:

env:

```
- name: SA_PASSWORD  
  value: "P@ssword123"
```

Correct Way — Kubernetes Secret

```
kubectl create secret generic sql-secret \  
  --from-literal=SA_PASSWORD='Strong@Pass123'
```

env:

```
- name: SA_PASSWORD  
  valueFrom:  
    secretKeyRef:  
      name: sql-secret  
      key: SA_PASSWORD
```

Bonus (Production)

Use:

- Azure Key Vault + CSI Driver
- AWS Secrets Manager
- HashiCorp Vault

.NET Concepts for Backend Developer

2. TLS Certificates (HTTPS Everywhere)

Install cert-manager

```
kubectl apply -f https://github.com/cert-manager/cert-manager/releases/latest/download/cert-manager.yaml
```

Create Issuer

```
apiVersion: cert-manager.io/v1
kind: ClusterIssuer
metadata:
  name: letsencrypt
spec:
  acme:
    server: https://acme-v02.api.letsencrypt.org/directory
    email: admin@myenterprise.com
    privateKeySecretRef:
      name: letsencrypt-key
    solvers:
      - http01:
          ingress:
            class: nginx
```

Secure Ingress

```
annotations:
  cert-manager.io/cluster-issuer: letsencrypt
spec:
  tls:
    - hosts:
        - api.myenterprise.com
      secretName: api-tls
```

3. Network Policies (Zero Trust Inside Cluster)

```
apiVersion: networking.k8s.io/v1
```

.NET Concepts for Backend Developer

kind: NetworkPolicy

metadata:

name: allow-api-only

spec:

podSelector:

matchLabels:

app: api

ingress:

- from:

- podSelector:

matchLabels:

app: gateway

Now **only Gateway can talk to API**.

4. Pod Security (Run as Non-Root)

securityContext:

runAsNonRoot: true

runAsUser: 1000

Drop Linux Capabilities

securityContext:

capabilities:

drop: ["ALL"]

5. Container Hardening

Dockerfile:

FROM mcr.microsoft.com/dotnet/aspnet:8.0-alpine

RUN adduser -D appuser

USER appuser

6. RBAC — Least Privilege

kind: Role

.NET Concepts for Backend Developer

rules:

- apiGroups: [""]

resources: ["pods"]

verbs: ["get", "list"]

7. Admission Policies (Prevent Bad Deploys)

Use **OPA Gatekeeper**:

Prevent:

- Privileged containers
 - Latest tags
 - No resource limits
 - No probes
-

8. Observability Security

- Audit logs enabled
 - Prometheus secured by auth
 - Grafana behind OAuth
 - Alert on:
 - Failed logins
 - Pod restarts
 - 5xx spikes
 - CPU anomalies
-

9. CI/CD Security

Add:

- name: Scan Image

uses: aquasecurity/trivy-action@master

Block pipeline on high vulnerabilities.

Final Production Readiness Score

.NET Concepts for Backend Developer

Area	Status
Secrets	✓ Encrypted
TLS	✓ Auto-managed
Network Isolation	✓
RBAC	✓
Container Hardening	✓
Zero Trust	✓
Auditing	✓
Vulnerability Scanning	✓

You now have the **same security posture as banks & fintech platforms**.

Auto rollback on failure

Auto Rollback on Failure is the heart of real production reliability.

.NET 8 + Kubernetes + Canary + Prometheus + GitHub Actions stack.

Auto Rollback Architecture

Deploy Canary → Observe Metrics → Decision Engine → Promote OR Rollback

Step 1 — Health Endpoint in .NET 8

```
builder.Services.AddHealthChecks();
```

```
app.MapHealthChecks("/health");
```

Step 2 — Prometheus Metrics

Add OpenTelemetry metrics + Prometheus exporter.

Prometheus tracks:

Metric	Why
http_requests_total	Error rate
http_request_duration_seconds	Latency
process_cpu_seconds_total	Resource stress

.NET Concepts for Backend Developer

Step 3 — Alert Rules

prometheus-alerts.yaml

groups:

- name: canary-health

rules:

- alert: CanaryHighErrorRate

expr: rate(http_requests_total{job="api-canary",status=~"5.."}[1m]) > 0.05

for: 2m

Step 4 — GitHub Action Auto Decision

Add this to your pipeline:

auto-rollback:

needs: deploy-canary

runs-on: ubuntu-latest

steps:

- name: Wait for metrics

run: sleep 120

- name: Query Prometheus

run: |

```
ERROR_RATE=$(curl -s http://prometheus.monitoring:9090/api/v1/query \
--data-urlencode 'query=rate(http_requests_total{job="api-canary",status=~"5.."}[1m])')
```

```
if echo "$ERROR_RATE" | grep -q '"value":[^\,]*,[0-9]\.'; then
```

```
  echo "Rollback triggered"
```

```
  kubectl scale deployment api-canary --replicas=0
```

```
  kubectl scale deployment api-stable --replicas=10
```

```
  exit 1
```

```
fi
```

.NET Concepts for Backend Developer

Step 5 — Promotion Path

If no alert triggers:

promote:

needs: auto-rollback

runs-on: ubuntu-latest

steps:

- run: |

kubectl scale deployment api-stable --replicas=0

kubectl scale deployment api-canary --replicas=10

Step 6 — Kubernetes Safety Net

strategy:

type: RollingUpdate

rollingUpdate:

maxUnavailable: 0

maxSurge: 1

Plus readiness probes:

readinessProbe:

httpGet:

path: /health

port: 5000

initialDelaySeconds: 5

Result

Feature	Achieved
Automatic detection	✓
Zero-downtime rollback	✓
Self-healing system	✓
Human-free recovery	✓
Enterprise-grade reliability	✓

This is exactly how Netflix, Stripe, Uber, and Amazon protect production.